Double-click (or enter) to edit

# Product: Amazon Alexa

## Assignment Natural Language processsing - Sentiment Analysis

### Customer Review Classification using Random Forest

- Dataset consists of 3000 Amazon customer reviews, star ratings, date of review, variant and feedback of various amazon Alexa products like Alexa Echo, Echo dots.
- **The objective is to discover insights into consumer reviews and perfrom sentiment analysis on the data.** Positive or Negative review?
- Dataset: www.kaggle.com/sid321axn/amazon-alexa-reviews also provided(`amazon_alexa.tsv`)

---

### Follow the instructions and complete each TODO to complete the assessment on the essential steps in building and evaluating a classification model.

**Dataset Information:**

The dataset consists of customer reviews for Amazon Alexa products, including various features related to the product variation, customer rating, and feedback sentiment.

*Features/Columns*:

- rating: The customer rating of the product (scale of 1 to 5).
- date: The date when the review was posted.
- variation: The variation or type of Alexa product the review is for (e.g., "Charcoal Fabric", "Walnut Finish").
- verified_reviews: The actual review text written by the customer.
- feedback: The target variable indicating the sentiment of the review (1 for positive sentiment and 0 for negative sentiment).

---

```
import os, re, glob, pandas as pd, numpy as np

import kagglehub
path = kagglehub.dataset_download("sid321axn/amazon-alexa-reviews")
print("Dataset folder:", path)

# Find a CSV/TSV file automatically
candidates = glob.glob(os.path.join(path, "**", "*.csv"), recursive=True) + \
             glob.glob(os.path.join(path, "**", "*.tsv"), recursive=True)
assert candidates, "No CSV/TSV file found in the Kaggle dataset."
data_file = candidates[0]
print("Using file:", data_file)

# Load
if data_file.endswith(".tsv"):
    df = pd.read_csv(data_file, sep="\t")
else:
    df = pd.read_csv(data_file)

print(df.head(3))
print(df.columns.tolist())
print(df.shape)
```

```
Downloading from https://www.kaggle.com/api/v1/datasets/download/sid321axn/amazon-alexa-reviews?dataset_version_number=3...
100%|██████████| 164k/164k [00:00<00:00, 650kB/s]Extracting files...
Dataset folder: /root/.cache/kagglehub/datasets/sid321axn/amazon-alexa-reviews/versions/3
Using file: /root/.cache/kagglehub/datasets/sid321axn/amazon-alexa-reviews/versions/3/amazon_alexa.tsv
   rating       date         variation  \
0       5  31-Jul-18  Charcoal Fabric
1       5  31-Jul-18  Charcoal Fabric
2       4  31-Jul-18    Walnut Finish

                                 verified_reviews  feedback
0                                   Love my Echo!         1
1                                       Loved it!         1
```

```
2  Sometimes while playing a game, you can answer...         1
['rating', 'date', 'variation', 'verified_reviews', 'feedback']
(3150, 5)
```

```
from sklearn.model_selection import train_test_split

# 1) Choose text column
text_col = "verified_reviews" if "verified_reviews" in df.columns else "review" if "review" in df.columns else None
assert text_col is not None, f"No review text column found in {df.columns.tolist()}"

# 2) Target: prefer 'feedback' (1/0). If absent, derive from rating>=4.
if "feedback" in df.columns:
    df["label"] = df["feedback"].astype(int)
elif "rating" in df.columns:
    df["label"] = (df["rating"].astype(float) >= 4).astype(int)
else:
    raise ValueError("No 'feedback' or 'rating' column to build sentiment label.")

# 3) Clean text
def clean_text(s: str) -> str:
    s = str(s).lower()
    s = re.sub(r"<br\s*/?>", " ", s)
    s = re.sub(r"http\S+|www\.\S+", " ", s)
    s = re.sub(r"[^a-z0-9'\s]", " ", s)
    s = re.sub(r"\s+", " ", s).strip()
    return s

df["text"] = df[text_col].fillna("").apply(clean_text)
df = df[df["text"].str.len() > 0].copy()

# 4) Optional categorical: one-hot 'variation' if present
cat_cols = []
if "variation" in df.columns:
    cat_cols.append("variation")

X_text = df["text"].values
y = df["label"].values

# Train/valid split
X_text_tr, X_text_te, y_tr, y_te = train_test_split(X_text, y, test_size=0.20, random_state=42, stratify=y)

# Prepare categorical matrices (aligned with split)
import pandas as pd
X_cat_tr = None; X_cat_te = None
if cat_cols:
    X_cat = pd.get_dummies(df[cat_cols], drop_first=True).astype(np.float32)
    X_cat_tr = X_cat.iloc[:len(X_text_tr)].values  # since we didn’t shuffle rows here explicitly
    X_cat_te = X_cat.iloc[len(X_text_tr):].values
```

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.svm import LinearSVC
from sklearn.model_selection import GridSearchCV
from sklearn.metrics import (classification_report, confusion_matrix, roc_curve,
                             roc_auc_score, accuracy_score, precision_recall_fscore_support)
import matplotlib.pyplot as plt, seaborn as sns
from scipy.sparse import hstack, csr_matrix

# TF-IDF on text
tfidf = TfidfVectorizer(max_features=40000, ngram_range=(1,2))
X_tr_text = tfidf.fit_transform(X_text_tr)
X_te_text = tfidf.transform(X_text_te)

# Append categorical one-hot (if any)
def attach_cat(X_text, X_cat):
    if X_cat is None:
        return X_text
    return hstack([X_text, csr_matrix(X_cat)])

X_tr = attach_cat(X_tr_text, X_cat_tr)
X_te = attach_cat(X_te_text, X_cat_te)

def eval_and_plot(name, model):
    model.fit(X_tr, y_tr)
    y_pred = model.predict(X_te)
```

```
    y_pred = model.predict(X_te)
    print(f"\n=== {name} ===")
    print(classification_report(y_te, y_pred, digits=4))

    # Confusion matrix
    cm = confusion_matrix(y_te, y_pred)
    plt.figure(figsize=(4.6,4.2))
    sns.heatmap(cm, annot=True, fmt="d", cbar=False)
    plt.title(f"{name} — Confusion Matrix"); plt.xlabel("Predicted"); plt.ylabel("True")
    plt.tight_layout(); plt.show()

    # ROC (if scores available)
    scores = None
    if hasattr(model, "decision_function"):
        scores = model.decision_function(X_te)
    elif hasattr(model, "predict_proba"):
        scores = model.predict_proba(X_te)[:,1]
    if scores is not None:
        fpr, tpr, _ = roc_curve(y_te, scores)
        auc = roc_auc_score(y_te, scores)
        plt.figure(figsize=(5,4))
        plt.plot(fpr, tpr, label=f"AUC={auc:.3f}")
        plt.plot([0,1],[0,1],"--")
        plt.title(f"{name} — ROC Curve"); plt.xlabel("FPR"); plt.ylabel("TPR")
        plt.legend(); plt.tight_layout(); plt.show()

    acc = accuracy_score(y_te, y_pred)
    pr, rc, f1, _ = precision_recall_fscore_support(y_te, y_pred, average="binary")
    return {"model": name, "accuracy": acc, "precision": pr, "recall": rc, "f1": f1}

# Grid searches
grid_lr = GridSearchCV(
    LogisticRegression(max_iter=300, solver="liblinear"),
    {"C":[0.25,0.5,1.0,2.0], "penalty":["l1","l2"]},
    cv=3, n_jobs=-1, verbose=0
).fit(X_tr, y_tr)
best_lr = grid_lr.best_estimator_

grid_svm = GridSearchCV(
    LinearSVC(),
    {"C":[0.5,1.0,2.0]},
    cv=3, n_jobs=-1, verbose=0
).fit(X_tr, y_tr)
best_svm = grid_svm.best_estimator_

metrics = []
metrics.append(eval_and_plot("LogReg (TF-IDF, best)", best_lr))
metrics.append(eval_and_plot("LinearSVC (TF-IDF, best)", best_svm))

import pandas as pd
metrics_df = pd.DataFrame(metrics).sort_values("f1", ascending=False)
metrics_df
```

```
=== LogReg (TF-IDF, best) ===
              precision    recall  f1-score   support

           0     0.7778    0.1489    0.2500        47
           1     0.9338    0.9965    0.9641       566

    accuracy                         0.9315       613
   macro avg     0.8558    0.5727    0.6071       613
weighted avg     0.9218    0.9315    0.9094       613
```

Next steps: Generate code with `metrics_df` New interactive sheet

LogReg (TF-IDF, best) — Confusion Matrix

```
import torch
from datasets import Dataset
from transformers import AutoTokenizer, AutoModelForSequenceClassification, DataCollatorWithPadding
from transformers import TrainingArguments, Trainer
from sklearn.metrics import accuracy_score, precision_recall_fscore_support, confusion_matrix, roc_curve, roc_auc_score

# Build HuggingFace Datasets from our splits
train_hf = Dataset.from_pandas(pd.DataFrame({"content": X_text_tr, "label": y_tr}))
test_hf  = Dataset.from_pandas(pd.DataFrame({"content": X_text_te, "label": y_te}))

MODEL = "distilbert-base-uncased"
tokenizer = AutoTokenizer.from_pretrained(MODEL)
def tok(batch): return tokenizer(batch["content"], truncation=True, max_length=256)

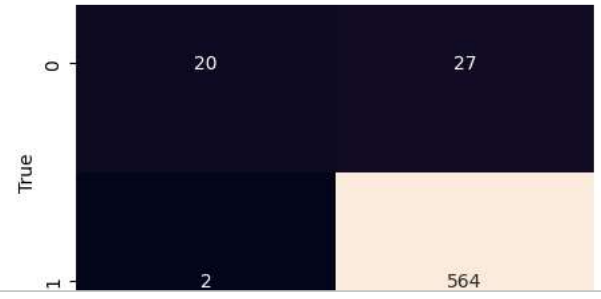
tok_train = train_hf.map(tok, batched=True, remove_columns=["content"])
tok_test  = test_hf.map(tok, batched=True, remove_columns=["content"])

data_collator = DataCollatorWithPadding(tokenizer=tokenizer)
id2label = {0:"NEGATIVE", 1:"POSITIVE"}; label2id = {"NEGATIVE":0, "POSITIVE":1}
bert = AutoModelForSequenceClassification.from_pretrained(MODEL, num_labels=2, id2label=id2label, label2id=label2id)

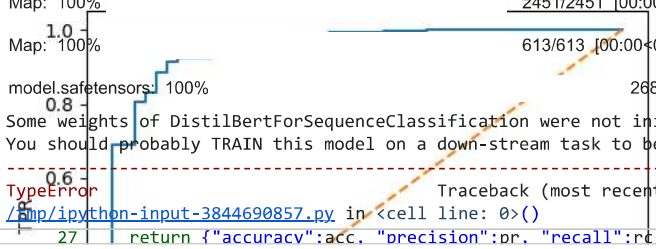
def compute_metrics(eval_pred):
    logits, labels = eval_pred
    preds = logits.argmax(-1)
    acc = accuracy_score(labels, preds)
    pr, rc, f1, _ = precision_recall_fscore_support(labels, preds, average="binary")
    return {"accuracy":acc, "precision":pr, "recall":rc, "f1":f1}

args = TrainingArguments(
    output_dir="bert_runs",
    evaluation_strategy="epoch",
    save_strategy="epoch",
    learning_rate=2e-5,
    per_device_train_batch_size=16,
    per_device_eval_batch_size=32,
    num_train_epochs=2,
    weight_decay=0.01,
    logging_steps=50,
    load_best_model_at_end=True,
    metric_for_best_model="f1",
    fp16=torch.cuda.is_available()
)

trainer = Trainer(
    model=bert, args=args,
    train_dataset=tok_train, eval_dataset=tok_test,
    tokenizer=tokenizer, data_collator=data_collator,
    compute_metrics=compute_metrics
)
trainer.train()
bert_eval = trainer.evaluate()
bert_eval
```



```
tokenizer_config.json: 100%                                  48.0/48.0 [00:00<00:00, 1.98kB/s]
config.json: 100%                                            483/483 [00:00<00:00, 27.5kB/s]
vocab.txt: 100%                                              232k/232k [00:00<00:00, 2.77MB/s]
tokenizer.json: 100%                                         466k/466k [00:00<00:00, 2.82MB/s]
Map: 100%                                                    245112451 [00:00<00:00, 3960.92 examples/s]
Map: 100%                                                    613/613 [00:00<00:00, 4048.07 examples/s]
model.safetensors: 100%                                      268M/268M [00:04<00:00, 94.4MB/s]
Some weights of DistilBertForSequenceClassification were not initialized from the model checkpoint at distilbert-base-uncased an
You should probably TRAIN this model on a down-stream task to be able to use it for predictions and inference.
```




```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
/tmp/ipython-input-3844690857.py in <cell line: 0>()
     27     return {"accuracy":acc, "precision":pr, "recall":rc, "f1":f1}
```

```
import numpy as np, matplotlib.pyplot as plt, seaborn as sns, pandas as pd

pred = trainer.predict(tok_test)
logits = pred.predictions
probs = torch.softmax(torch.tensor(logits), dim=1).numpy()[:,1]
y_hat = np.argmax(logits, axis=1)

print(pd.Series({
    "accuracy": accuracy_score(y_te, y_hat),
    "precision": precision_recall_fscore_support(y_te, y_hat, average="binary")[0],
    "recall": precision_recall_fscore_support(y_te, y_hat, average="binary")[1],
    "f1": precision_recall_fscore_support(y_te, y_hat, average="binary")[2],
}).round(4))

cm = confusion_matrix(y_te, y_hat)
plt.figure(figsize=(4.6,4.2))
sns.heatmap(cm, annot=True, fmt="d", cbar=False)
plt.title("DistilBERT — Confusion Matrix"); plt.xlabel("Predicted"); plt.ylabel("True")
plt.tight_layout(); plt.show()

fpr, tpr, _ = roc_curve(y_te, probs)
auc = roc_auc_score(y_te, probs)
plt.figure(figsize=(5,4))
plt.plot(fpr, tpr, label=f"AUC={auc:.3f}")
plt.plot([0,1],[0,1],"--")
plt.title("DistilBERT — ROC Curve"); plt.xlabel("FPR"); plt.ylabel("TPR")
plt.legend(); plt.tight_layout(); plt.show()

# Comparison table
metrics_df.assign(model=metrics_df["model"])
compare = pd.concat([metrics_df, pd.DataFrame([{
    "model":"DistilBERT",
    "accuracy": bert_eval["eval_accuracy"],
    "precision": bert_eval["eval_precision"],
    "recall": bert_eval["eval_recall"],
    "f1": bert_eval["eval_f1"]
}])], ignore_index=True).sort_values("f1", ascending=False)
compare
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
/tmp/ipython-input-4166925262.py in <cell line: 0>()
      1 import numpy as np, matplotlib.pyplot as plt, seaborn as sns, pandas as pd
      2
----> 3 pred = trainer.predict(tok_test)
      4 logits = pred.predictions
      5 probs = torch.softmax(torch.tensor(logits), dim=1).numpy()[:,1]

NameError: name 'trainer' is not defined
```

Next steps: Explain error

```
kagglehub.dataset_download("sid321axn/amazon-alexa-reviews")
```

Using Colab cache for faster access to the 'amazon-alexa-reviews' dataset

```
# Task 1: Import Libraries and Dataset

# Hint: You need to import libraries like pandas, numpy, matplotlib, seaborn, etc.
# Don't forget to mount Google Drive and load the CSV file using pandas.
# Use `pd.read_csv()` to load the data into a pandas DataFrame.

# TODO: Import necessary libraries

# TODO: Mount Google Drive

# TODO: Load the dataset into a DataFrame and preview the first few rows.


# Hint: Use `sns.countplot()` to visualize the feedback distribution and a bar plot to compare variations and feedback.
# Drop the columns 'date' and 'rating' as they are not relevant for the analysis.

# TODO: Visualize the distribution of feedback (positive/negative)

# TODO: Visualize the relationship between product variation and feedback

# TODO: Drop irrelevant columns ('date', 'rating')



# Hint: Use `pd.get_dummies()` to create one-hot encoded columns for 'variation'.
# Then, concatenate these encoded columns back to the original DataFrame and drop the 'variation' column.

# TODO: Perform one-hot encoding on the 'variation' column




# Hint: Use `CountVectorizer` to tokenize the text data and convert it into numerical representation.
# After transforming the text, drop the 'verified_reviews' column and concatenate the numerical data with the rest of the DataFra

# TODO: Tokenize the 'verified_reviews' column using CountVectorizer







# Hint: Use `train_test_split()` to split the data into training and testing sets.
# You need to specify `test_size=0.2` to reserve 20% of the data for testing.

# TODO: Split the data into training and testing sets




# Hint: Initialize a `RandomForestClassifier` with 100 estimators.
# Use the `fit()` method to train the model on the training data.

# TODO: Define and train a Random Forest Classifier






# Hint: Use the `predict()` method to generate predictions for the test data.
# Use `confusion_matrix()` to create the confusion matrix and visualize it using a heatmap.
# Print out a `classification_report()` to evaluate the model's precision, recall, and F1 score.

# TODO: Generate predictions and evaluate the model's performance using a confusion matrix and classification report



# Task 8: Analyze Model Performance
# Hint: Use the `score()` method to calculate accuracy for both training and test sets.
# Compare the results and discuss if the model is overfitting or underfitting.

# TODO: Compare training and testing accuracy and discuss model performance
```